

ALGORITHMIC STORAGE: THE ALGORITHMICDISK ON THE FOOTPRINT–COMPUTE–HORIZON TRIANGLE

A Seed-Driven Storage Device, Powered by the AlgorithmicNET Generator,
that Trades Processing Power and Time for Storage Space

Milind K. Patil
Syncaissa Systems Inc.
syncaissa@outlook.com

September 2026

Abstract

Algorithmic Storage is a paradigm in which data is *regenerated* rather than retained: a spatial cost, storage, is converted into a temporal one, regeneration time. Its device, the **AlgorithmicDisk**, holds only shared generator engines and one small **Seed** per object, and regenerates any object on demand, bit-exactly, by running an engine—the **AlgorithmicNET**—forward from the Seed. The three resources a deployment spends form the **Regeneration Triangle**: Footprint (bytes at rest), Compute (processing capacity) and Horizon (time allowed); reducing one side must be paid for by the other two, and the object’s information content is a floor below which no trade goes. A reference implementation measures the trade end to end. Shared by four thousand text objects, a 26 KB engine drives the per-object Footprint from $4.2\times$ to $0.66\times$ the raw size as the corpus grows (**gzip** over the whole corpus: $0.35\times$); a 148 KB engine stores two books it never saw at $0.27\times$ and $0.26\times$, below **xz** and **bz2** but above PPMd, the classical text compressor; a revised document costs its edits alone ($0.002\times$); a library document costs a 10-byte reference; a thousand program-generated 105 MB synthetic-telemetry files—104 GB—are stored in 317 KB and every one regenerated bit-exactly; an integer-only engine, implemented in Python and C, produces identical bytes under every change of arithmetic tested. The same tests report where the trade fails: already-compressed data, and files in no domain the disk knows, are stored raw, and on novel text the engine is a mid-range compressor, not a record. The law, the device, the engines, and the measurements are the contribution; the code, engines and disks are released.

Keywords: Algorithmic Storage, AlgorithmicDisk, AlgorithmicNET, regenerative storage, Regeneration Triangle, Footprint, Compute, Horizon, seed-driven generation, deterministic generator, logical depth, Kolmogorov complexity, time–space trade-off, deep-space data delivery, millennial archive

Key Insight

Results at a glance. Every number below is bit-exact regeneration from Seeds and shared engines, measured on one CPU core; every table in Section 5 is generated from the released results. **Where the disk holds the generator:** 1,000 telemetry files, 104 GB, from 317 KB (3×10^{-6}); a 3.5 MB animation from a 110-byte Seed. **Where it holds a base version or a library:** a revision for 390 bytes ($0.002\times$); a library document for 10 bytes. **Where it knows the domain:** unseen books at $0.27\times$, below `xz -9` and `bz2` but 4% above PPMd, which needs no engine at all (Table 8). **Where it knows nothing:** a PNG, an MP4 and Flate-compressed PDFs at $1.000\times$ Seed ($1.005\text{--}1.019\times$ with block headers and the engine’s share)—never materially worse than raw, and there `xz` wins. On a deliberately mixed set of ten files the disk is behind `xz` (0.49 against 0.42), which is the honest shape of the paradigm: orders of magnitude where the disk knows how the data was made, parity where it knows the language, nothing where it knows neither.

1 Introduction

Storage systems are designed around one scarce resource, space on a medium. Compression (Shannon, 1948) removes statistical redundancy, but once an object is near its entropy the bytes must be kept. Several important deployments—interplanetary spacecraft, planetary outposts, thousand-year archives, palm-scale field devices, and schools beyond the reach of the internet—are short of exactly that resource while rich, by terrestrial standards absurdly rich, in two others: a probe on a decades-long cruise has time; a solar-powered outpost has energy-over-time, hence compute; an archive read once a decade can afford hours per retrieval. Storage systems accept one currency, bytes; these systems have plenty of two other currencies—processing power and time—and almost none of the first.

Algorithmic Storage lets a system pay for data in whichever currency it has in abundance. It converts a spatial cost into a temporal one: the bytes an object would occupy on a medium become the seconds a device spends regenerating it, plus the one-time cost of the engine that knows how. The conversion has an exchange rate, set by how well the engine knows the object, and a floor, the object’s own information, below which no amount of time buys any more space. It is the classical space–time trade-off (Hellman, 1980; Chen et al., 2016) applied to data at rest, with a Manifest as the receipt. Instead of storing an object $\mathbf{x}$, the **AlgorithmicDisk** stores a compact Seed $\mathbf{s}$ and shares a deterministic generator, the **AlgorithmicNET**, with weights θ ; reading runs the generator forward from the Seed; writing discovers the Seed.

Contributions. (1) The Regeneration Triangle and its conservation law, stated through the work-bounded description length $\mathcal{F}^*(\mathcal{W})$ (Section 2). (2) The AlgorithmicDisk and its engine: a seeded, recurrent, deterministic generator with shared weights, block-parallel manifestation, a two-phase write/read protocol, and a Manifest that makes every object verifiable anywhere the engine exists (Sections 3–4). (3) Measured results over the whole trade—amortisation, whole files, a multi-engine disk with procedural, library, delta and retrieval engines, 100 MB-class deep data, a thousand-file proof, and a ten-file end-to-end demonstration (Section 5). (4) Determinism: an integer-only engine, implemented twice with byte-identical output (Section 5.5). (5) The appliance that realises the disk as hardware, deployment regimes and a regenerate-versus-cache policy (Section 6); open experiments and parallel emission schedules as future work (Section 7).

2 The Regeneration Triangle and Its Law

The three names are nested: Algorithmic Storage is the paradigm, the AlgorithmicDisk the device that realises it, the AlgorithmicNET the generator engine inside the device. The three resources are **Footprint** $\mathcal{F}$ (bits at rest), **Compute** $\mathcal{C}$ (operations per second) and **Horizon** $\mathcal{H}$ (seconds allowed per regeneration); their product $\mathcal{W} = \mathcal{C} \cdot \mathcal{H}$ is the Work available (Table 1, Figure 1).

Table 1: The three sides of the Regeneration Triangle.

Side	Symbol	Unit	Measures	Scarce in
Footprint	$\mathcal{F}$	bits / bytes	storage at rest	spacecraft, archives, palm devices
Compute	$\mathcal{C}$	operations / s	processing capacity of the regenerator	probes, low-power nodes
Horizon	$\mathcal{H}$	seconds	time allowed per regeneration	data centres, interactive use
Work	$\mathcal{W} = \mathcal{C} \cdot \mathcal{H}$	operations	computation spent per regeneration	—

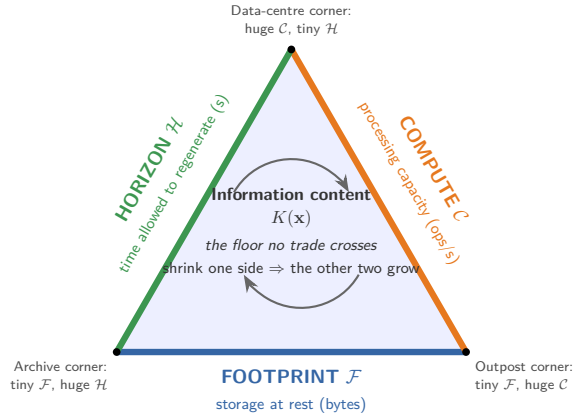


Figure 1: The **Regeneration Triangle**. Each side is one system resource; at the centre is the information content $K(\mathbf{x})$ of the stored object, the floor below which no trade goes. A deployment lives at the corner where its two abundant resources meet, and pays with them for the scarce third side. Work $\mathcal{W} = \mathcal{C} \cdot \mathcal{H}$ is the product of the two non-storage sides.

Key Insight

Why a triangle. The triangle is a mnemonic, not a geometric model: three resources, one object, one floor. A trade between two resources is a curve; among three it is a surface, and the figure names its axes and the floor in the middle. The mathematics is in Proposition 2.1, whose content is elementary and is collected there for reference, not claimed as a theorem. Every deployment in this paper is a point on that surface.

Definition 2.1 (AlgorithmicDisk and AlgorithmicNET). An AlgorithmicDisk is $\mathcal{D} = (\{\theta_g\}_g, \{\text{RM}_i\}_i, \text{cache})$: generator weight sets indexed by identity g , a catalogue of Regeneration Manifests, and an optional cache of materialised objects; its *apparent capacity* is $\sum_i N_i$ and its *resident Footprint* is $\sum_g |\theta_g| + \sum_i |\text{RM}_i| + |\text{cache}|$. An AlgorithmicNET is a tuple $(\theta, \Phi_\theta, \psi, \mathbf{s})$: shared weights, a deterministic state transition, a deterministic read-out emitting b bits per step, and a Seed $\mathbf{s} \in \{0, 1\}^k$. Its *manifestation* is

$$\begin{aligned} \text{Man}_\theta(\mathbf{s}) &= \mathbf{x}_1 \parallel \mathbf{x}_2 \parallel \cdots, & \mathbf{x}_t &= \psi(\mathbf{h}_t), \\ \mathbf{h}_t &= \Phi_\theta(\mathbf{h}_{t-1}, \mathbf{x}_{t-1}, \mathbf{s}, t), & \mathbf{h}_0 &= \mathbf{0}, \end{aligned} \quad (1)$$

with two feedback paths—the previous state and the bits the engine itself emitted—and the

Seed injected at every step. For a *predictive* engine the state defines a distribution over $\mathbf{x}_t$ and ψ consumes Seed bits through an arithmetic decoder to select $\mathbf{x}_t$: the same equation with ψ stateful in $\mathbf{s}$. An object $\mathbf{x}$ is stored if $\text{Man}_{\theta}(\mathbf{s})_{1:N} = \mathbf{x}$. The *Regeneration Manifest* of $\mathbf{x}$ is $\text{RM}(\mathbf{x}) = (\text{GID}, \mathbf{s}, N, \text{hash}(\mathbf{x}))$, where $\text{GID} = \text{hash}(\theta, \Phi, \psi)$ pins the exact generator; any device holding that generator regenerates the object bit-for-bit and proves it with the digest. The Manifest is the unit of transmission, of archival and of deduplication (identical Manifests are identical objects), and a catalogue of Manifests is an index that can be queried without regenerating anything.

The picture to hold in mind is this: an object is treated as if it were the output of a generator, and instead of storing its bytes the disk stores the generator’s identity and the Seed that drives the generator to emit exactly those bytes. Whether that Seed is short is not a matter of cleverness in the generator’s stream but of knowledge: a file that is random *to* the generator needs a Seed as long as itself (the pigeonhole principle, measured in Table 4), and a file the generator knows how to produce needs only what the generator cannot guess. Every engine in this paper is a way of making files less random to the disk.

Definition 2.2 (Footprint and the three Works). For an object whose engine is shared by M objects, $\mathcal{F}(\mathbf{x}) = |\mathbf{s}| + |\theta|/M$. Work is spent at three moments and the paper keeps them apart: $\mathcal{W}_{\text{read}}$ (Manifestation), $\mathcal{W}_{\text{write}}$ (Seed Discovery) and $\mathcal{W}_{\text{train}}$ (building an engine, once per domain). An object is *deep* relative to a disk if some engine gives it a Footprint far below what a general-purpose compressor finds, because its regularity is computational—it is the output of a generator—rather than statistical; *shallow* if no engine beats the raw bytes by more than such a compressor does. Bennett’s logical depth (Bennett, 1988) is the limiting case in which regeneration Work, not only Footprint, separates the two.

Let $K(\mathbf{x})$ be the Kolmogorov complexity of $\mathbf{x}$ (Kolmogorov, 1965; Chaitin, 1966; Solomonoff, 1964; Li and Vitányi, 2008) and $K^{\mathcal{W}}(\mathbf{x})$ its work-bounded version (Levin, 1973; Hartmanis, 1983). The *regeneration frontier* is $\mathcal{F}^*(\mathcal{W}) = K^{\mathcal{W}}(\mathbf{x})$, the smallest Footprint from which $\mathbf{x}$ regenerates with Work at most $\mathcal{W}$. The word *conservation* below is an analogy and should not be over-read: nothing is exchanged one-for-one, and $K(\mathbf{x})$ is a floor, not a conserved quantity.

Proposition 2.1 (Conservation law of the Regeneration Triangle). *For every object $\mathbf{x}$ with $|\mathbf{x}| = N$: (i) Trade: $\mathcal{F}^*(\mathcal{W})$ is non-increasing in $\mathcal{W}$. (ii) Ceiling: $\mathcal{F}^*(\mathcal{W}) \leq N + O(1)$ for all $\mathcal{W} \geq cN$ —the object can always be kept verbatim. (iii) Floor: $\mathcal{F}^*(\mathcal{W}) \geq K(\mathbf{x})$ for all $\mathcal{W}$, with equality in the limit. (iv) Exchange: $\mathcal{C}$ and $\mathcal{H}$ are substitutable through $\mathcal{W} = \mathcal{C} \cdot \mathcal{H}$ whenever the generator is block-parallelisable.*

Proof. (i) A program valid under budget $\mathcal{W}$ is valid under any larger budget. (ii) “Print $\mathbf{x}$ ” has length $N + O(1)$ and linear work. (iii) A work-bounded program is a program, so its length is at least $K(\mathbf{x})$, attained once $\mathcal{W}$ exceeds the minimal program’s own running time. (iv) If block j depends only on $(\mathbf{s}, \theta, j)$, p processors of rate $\mathcal{C}/p$ finish in the Horizon of one processor of rate $\mathcal{C}$. $\square$

The floor is the “nothing is free” clause. It does not limit how much deep data can save—the gap between N and $K(\mathbf{x})$ is twelve orders of magnitude for a simulation dump—but it says which data the trade applies to; both ends are measured in Section 5 (Tables 9 and 6). Schematically, $\mathcal{F}^*(\mathcal{W})$ falls from the ceiling toward the floor as Work grows, steeply for deep objects and hardly at all for shallow ones; Table 4 measures the write-side relative of this curve. **Amortisation**

follows at once: M objects sharing θ cost $|\theta| + \sum_i |s_i|$, so the per-object Footprint approaches $|s_i|$ as M grows (measured in Table 5).

3 The AlgorithmicDisk and Its Engine

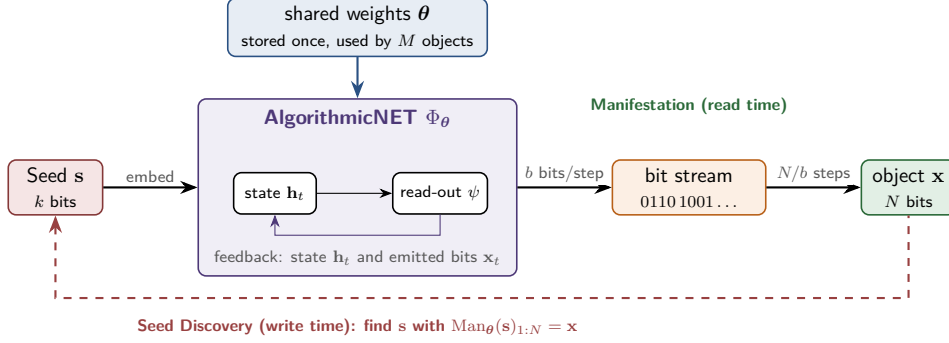


Figure 2: AlgorithmicNET architecture. Forward (solid): the Seed is embedded into the state; the generator applies the shared weights with feedback, and the read-out streams b bits per step until the N -bit object has been manifested. Inverse (dashed): at write time a search discovers the Seed that manifests the target object.

Generator cores. Three families sit at different points of the triangle. An *arithmetic* core (stream-cipher-like mixing, Bernstein, 2008) runs at gigabytes per second with tiny weights but its Seeds must be found by enumeration; a *neural* core learns a domain so that in-domain objects have short Seeds, discovered by gradient descent (Dupont et al., 2021; Sitzmann et al., 2020; Mildenhall et al., 2020; Patil, 2026); a *hybrid* core drives an arithmetic decoder with a learned distribution (Delétang et al., 2023), and its Seed is the entropy code. The paper’s measured engines are mostly of the last kind—the demonstration engine of Section 5.1 is a neural core with gradient discovery—together with procedural, delta and retrieval engines that need no network at all.

Block-parallel manifestation. To make Compute and Horizon substitutable (Proposition 2.1(iv)) the generator runs in counter mode: block j starts from $\mathbf{h} = \mathbf{0}$ with its own block code and sees the Seed at every step, so p cores manifest p blocks at once and one core manifests them in sequence over a longer Horizon; the stored description is unchanged. **Protocol.** Write: discover $\mathbf{s}$ (paying in Compute $\times$ Horizon), assert $\text{Man}_\theta(\mathbf{s})_{1:N} = \mathbf{x}$, store the Manifest. Read: require $\mathcal{C} \cdot \mathcal{H} \geq \mathcal{W}_{\text{read}}(N)$, stream the blocks in parallel over available cores, verify the digest, deliver or cache.

Trained like a network, deterministic like a program. Parts of what follows look like neural-network training, and a reader may ask whether such a system can give a deterministic answer. It can, and the distinction is the difference between a storage device and a generative model. Training is random—initialisation, data order—but its outcome is a fixed set of numbers, frozen, quantised to integers and named by a hash; from then on the engine is a *function*, and a retrained engine is a different engine with a different GID that no Manifest refers to. The predictive engine outputs probabilities, but the system never samples them: a chatbot draws its next token from the distribution it predicts, whereas here the distribution is a *code table* that the arithmetic decoder reads together with the Seed to return the one symbol the Seed dictates. The probability is a measuring instrument—it sets how many Seed bits a symbol costs—not a die. Search-found Seeds are stored only after the frozen engine has regenerated the object from them bit-for-bit; the integer-only engine of Section 5.5 removes floating-point summation order, the last

source of variation; and every Manifest carries a digest checked on every read. Randomness is spent once, at training, to choose the function; Manifestation never spends any (Table 2).

Table 2: Where randomness exists in the system and where it does not.

Phase	Random?	Consequence
Training an engine (once per domain)	yes: initialisation, data order	which engine you get; fixed by its GID
Seed Discovery by gradient search	yes: restarts	which Seed you get; verified before storing
Seed Discovery by entropy coding	no	one Seed per (engine, object)
Manifestation (every read)	no	the same bytes, every time, on every device

4 Seed Discovery

Seed Discovery is the inverse of manifestation and is where the triangle is paid for at write time. *Provenance capture*: the object was produced by a known seeded pipeline, so its parameters are recorded directly—cost near zero; simulation, procedural and synthetic data. *Gradient inversion*: freeze θ , optimise a continuous Seed against a reconstruction loss, quantise, verify—pays in Compute. *Entropy coding*: for a predictive engine the Seed is the arithmetic code of the object (Witten et al., 1987), produced by a 32-bit carry-less range coder of Subbotin’s design (Subbotin, 1999; Martin, 1979)—no search at all, one forward pass, and $|\mathbf{s}| = -\log_2 P_{\theta}(\mathbf{x})$, the object’s surprisal. *Enumerative search* (not implemented here) is the remaining option for hash-like engines.

Key Insight

Write once, pay once. In archival and deep-space regimes the write happens on Earth, in a data centre with effectively unlimited Compute; the read happens far away with effectively unlimited Horizon. Seed Discovery is therefore performed where Compute is cheap and Manifestation where time is cheap, and the Footprint that travels between them—the Manifest—is the only thing that must be small.

A k -bit Seed can address at most 2^k objects of a given length, so reachability is a design target: the engine is trained, or the pipeline chosen, so that the objects of interest fall inside the Seed space, and $k \geq K(\mathbf{x} \mid \theta)$ is the floor any design must respect.

5 Results

The reference implementation is NumPy on CPU (CuPy on GPU through a flag) with one C module; every timing is from one machine—an Intel Xeon Platinum 8259CL at 2.5 GHz exposing one physical core with two hardware threads, 8 GB of RAM, no GPU—called the test machine. Baselines are `gzip -9`, `bz2 -9` and `xz -9` unless a preset is given. All code, engines, disks and reference outputs are released, and each table can be re-created without training: `verify.py` opens every shipped disk, regenerates every object from its Seed with the shipped engines and checks the digest recorded in its Manifest; `demo/` and `demo2/` re-run the ten-file and thousand-file experiments; and `compare.py` diffs a fresh run against the reference results. The subsections follow the paradigm’s argument: an untrained engine and why it cannot win (5.1); a shared engine and the amortisation curve (5.2); whole files and an engine that beats `xz` (5.3); a disk of several

engines, with revisions and libraries (5.4); determinism (5.5); deep data at scale and the proof by a thousand files (5.6); ten files end to end (5.7); and what the numbers mean (5.8).

5.1 First object and the Seed-size sweep

The demo engine is a recurrent generator producing one character per step, every weight, block code, clock code and context sign-mask of which is generated from a 32-bit seed, so its weight Footprint is effectively zero:

$$\mathbf{h}_t^{(j)} = \tanh(\mathbf{W}_h \mathbf{h}_{t-1}^{(j)} + \mathbf{W}_x \mathbf{x}_{t-1}^{(j)} + \mathbf{W}_e (\mathbf{s} \odot \mathbf{m}_{j,t}) + \mathbf{p}_j + \mathbf{c}_t + \mathbf{b}_h), \quad \mathbf{x}_t^{(j)} = \mathbb{I}[\mathbf{W}_o \mathbf{h}_t^{(j)} + \mathbf{b}_o > 0]. \quad (2)$$

The first object stored on an AlgorithmicDisk was ‘The fox ran over the lazy dog’ (232 bits): a Seed of 256 dimensions at 4 bits was found by gradient inversion in 301 Adam iterations (0.5 s) and the object regenerated bit-exactly, block-parallel and sequentially, in an 834-byte disk file (Table 3). The Seed is 128 bytes for a 29-byte object, and Table 4 shows why: an untrained engine’s exchange rate is about one. As Seed dimensions rise, write-time Work falls monotonically (756→301→114→112 iterations)—the triangle in miniature—but a random map from a k -dimensional Seed can be expected to satisfy only of the order of $2k$ sign constraints (by analogy with Cover’s counting argument for linear threshold units, Cover, 1965), so $k = 128$ never reaches 232 bits. The gain must come from trained, shared weights.

Table 3: Round trip of the first object (gradient inversion on the untrained engine; free-running Manifestation; sha-256 verified).

Object	‘The fox ran over the lazy dog’ — 29 characters, $N = 232$ bits
Binary	01010100 01101000 01100101 00100000 01100110 ...
Generator identity	ANET-a02c1f203dc3721c ($k = 256$, hidden 256, 8 steps/block, 4 blocks)
Seed	256 dims $\times$ 4 bits = 1024 bits (128 bytes), found in 301 Adam iterations / 0.5 s
Manifestation	block-parallel 19 ms; sequential 7 ms; both bit-exact, sha-256 PASS
Disk file	demo.adisk: 834 bytes (generator spec 128 B + one Regeneration Manifest)

Table 4: Seed Discovery on the untrained engine for the 232-bit object (at most 3×2000 Adam iterations; “best” is the most bits matched when discovery failed).

k (dims)	bits/dim	Seed bits	ratio	exact	best	iterations
128	8	1024	4.4	no	219/232	6000
128	2	256	1.1	no	185/232	6000
256	8	2048	8.8	yes	232/232	756
256	4	1024	4.4	yes	232/232	301
256	2	512	2.2	no	207/232	6000
384	4	1536	6.6	yes	232/232	114
512	4	2048	8.8	yes	232/232	112
512	2	1024	4.4	no	229/232	6000

5.2 Amortisation: thousands of objects on one engine

Alice’s Adventures in Wonderland (Project Gutenberg #11) was cut into 4805 chunks of 232 bits, the last 200 held out. A first design trained the engine jointly with one latent Seed per chunk (teacher forcing (Williams and Zipser, 1989), straight-through quantisation (Bengio et al., 2013), Adam (Kingma and Ba, 2015)); on 500 chunks with 128-bit Seeds and a 128-unit engine, 3000 epochs drove the teacher-forced loss to 2×10^{-4} —essentially every bit satisfied when the engine

is fed the true previous byte—yet free-running regeneration from the quantised Seeds reached only 93.5 % bit accuracy and 0.8 % of objects exactly: the first wrong bit cascades through the output feedback (Ranzato et al., 2016), and 4-bit quantisation costs a further margin. It is kept as the fallback for engines that cannot express a distribution. The predictive design removes both problems: the engine predicts the next byte from the bytes it has emitted, is trained by maximum likelihood and frozen as int8 weights, and the Seed is the object’s arithmetic code.

Key Insight

Closed-form Seed Discovery. When the AlgorithmicNET is a predictor, the search for a Seed collapses to a computation: the Seed is the entropy code of the object under the engine, found in one forward pass, and its length is the object’s surprisal $|\mathbf{s}| = -\log_2 P_{\theta}(\mathbf{x})$ bits. Exactness is guaranteed by construction (entropy coding is lossless), the write path costs the same Work as the read path, and the Footprint of an object is exactly how unpredictable it is to the shared engine.

Table 5: Predictive engine on the Alice corpus: per-object Footprint against corpus size M (int8 weights amortised over M objects + mean Seed; “ideal” is the information-theoretic Seed length, “coded” the range-coder output with its 32-bit flush). All chunks regenerate bit-exactly; held-out chunks were never trained on.

hidden	M	weights (int8)	Seed coded	Seed ideal	held-out Seed	Footprint/obj	vs. raw	M^*
64	250	207,680 bits	140	114	142	971 bits	4.18	2262
64	500	207,680 bits	126	99	128	541 bits	2.33	1952
64	1000	207,680 bits	115	89	119	323 bits	1.39	1781
64	2000	207,680 bits	108	81	112	211 bits	0.91	1669
64	4000	207,680 bits	100	74	103	152 bits	0.66	1579
128	250	445,760 bits	128	101	132	1911 bits	8.24	4274
128	500	445,760 bits	118	91	123	1010 bits	4.35	3910
128	1000	445,760 bits	110	83	116	556 bits	2.39	3645
128	2000	445,760 bits	102	75	109	324 bits	1.40	3416
128	4000	445,760 bits	94	67	100	205 bits	0.88	3218

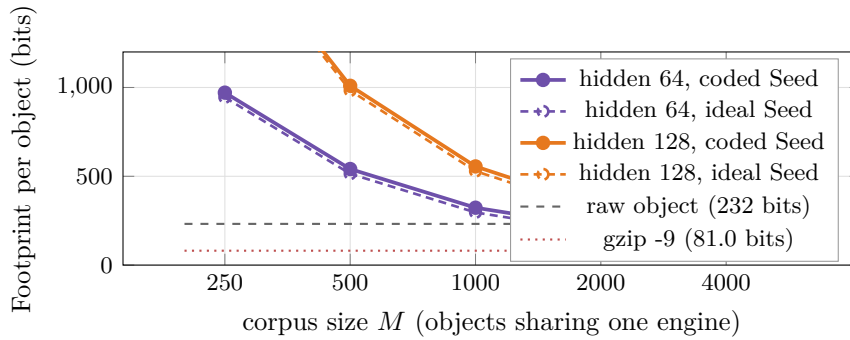


Figure 3: Amortisation curve of the predictive engine. The per-object Footprint is $|\theta|/M + |\mathbf{s}|$: the weight term vanishes as the corpus grows and the curve crosses below the raw object size at the break-even corpus size $M^* = |\theta|/(N - |\mathbf{s}|)$.

Every object, including the 200 never seen, regenerates bit-exactly. At $M = 4000$ the 64-unit engine holds 116 KB of text in 76 KB ($0.66\times$ raw; **gzip -9** over the whole corpus is $0.35\times$, so this is the amortisation mechanism demonstrated, not yet a win), crossing below the raw size at $M^* \approx 1600\text{--}3200$ objects; held-out Seeds are only 3–7 bits longer than training Seeds, so the engine learned the language, not the corpus. The Seed alone (67–94 bits ideal) is already at or below **gzip**’s 81 bits; what keeps the total above **gzip** on this small corpus is the weight term,

which halves every time the corpus doubles.

5.3 Whole files, and where a 148 KB engine stands among standard compressors

The predictive engine was trained on 2 MB of English (five books) and frozen as the shared generator: 55,720 bytes. Whole files are processed in 256-byte blocks from the zero state: *stream* mode runs one range coder over all blocks (smallest Seed, sequential read); *blocks* mode gives every block its own stream and batches the engine across blocks (block-parallel; Proposition 2.1(iv)) with a per-block *verbatim escape*—a block the engine cannot shorten is stored raw, so the Footprint never exceeds the file by more than two bytes per block. Table 6 stores four files as single objects.

Table 6: Real files as single objects with the 55 KB English engine (shared, counted once). Write = closed-form Seed Discovery, read = Manifestation; every regenerated file matched its sha-256. `pg12.txt` here is the raw Project Gutenberg file including its header and licence (196,534 B); Tables 9–7 use the book’s body alone (176,754 B).

file	bytes	gzip	xz	mode	Seed (B)	bits/byte	vs. xz	write (s)	read (s)
<code>pg12.txt</code>	196,534	69,779	61,752	stream	65,093	2.65	1.05	14	14
				blocks	69,183	2.82	1.12	12	13
<code>ideaBrainStormFile.txt</code>	42,425	16,910	15,860	stream	27,312	5.15	1.72	3	3
				blocks	27,637	5.21	1.74	2	3
<code>algorithmic_storage.py</code>	20,439	6,544	6,408	stream	15,901	6.22	2.48	2	2
				blocks	16,163	6.33	2.52	1	1
<code>image.png</code>	88,791	83,598	82,608	stream	153,831	13.86	1.86	7	7
				blocks	89,485	8.06	1.08	6	0

On the book the engine never saw (`pg12`, *Through the Looking-Glass*) the Seed is 2.65 bits per byte, 0.33×: below `gzip`, 5 % above `xz`. The PNG exposes both ends of the law: in stream mode the mismatched engine *expands* it to 1.73× (the floor made visible); in blocks mode the escape stores every block raw at 1.008× (the ceiling, implemented) and reads in 0.2 s. Out-of-domain text sits between, and there the engine loses to `gzip`. To test whether the loss to `xz` is a property of the approach or of the budget, a larger engine (hidden 256, 512-byte blocks, 148,024 bytes) was trained on 12 MB from sixteen texts (fifteen Gutenberg books and *Alice*) with two books held out entirely (Table 7); one caveat is that `pg12`, *Through the Looking-Glass*, is the sequel to *Alice* by the same author, so its style is not wholly unseen, whereas `pg84`, *Frankenstein*, has no relative in the corpus. Both come out *below* `xz` –9 and `bz2` –9, by 13 % and 18 % against `xz`, and the fixed-point engine of Section 5.5 gives up about 4 % of that and is still below `xz`. These are Seed sizes: the engine is paid once and, at a 5.4 % saving over `xz`, is repaid after about 2.8 MB of unseen books—roughly ten of this size.

Table 7: The larger English engine on two books it never saw. “float” is stream mode; “fixed/C” the integer engine in compiled blocks mode. All bit-exact.

book	bytes	gzip -9	bz2 -9	xz -9	Seed float	bits/byte	Seed fixed/C	bits/byte	read (s)
<code>pg12.txt</code>	176,754	62,949	49,248	55,736	48,391	2.190	50,143	2.270	6.3
<code>pg84.txt</code>	428,903	161,755	120,005	137,176	112,005	2.089	116,208	2.168	16.0

Beating `xz` is a low bar for text, and Table 8 adds the baselines a compression reviewer would ask for, on the same bytes. PPMd (Cleary and Witten, 1984; Shkarin, 2002), the classical context-model text compressor as shipped in 7-Zip, with *no* training data and no engine to store, is below the engine on both books, by 4 % over the two; given the same 12 MB corpus as a prior it is 12 % below. Brotli (Alakuijala and Szabadka, 2016), which ships a 122 KB static dictionary—the

closest existing analogue of a shared engine—and **zstd** with a 148,024-byte dictionary trained on the engine’s own corpus (Collet and Kucherawy, 2021) both stay above the engine. The fixed-point engine, which is what a device would carry, is beaten by **bz2** on **pg12**. The honest reading is that a 148 KB recurrent engine is a PPM-class text compressor, not a state-of-the-art one: the residue it leaves is about 2.1 bits per character where PPMd leaves 2.0 and cmix-class models about 1.2. What the engine offers that PPMd does not is not ratio but the device properties the rest of the paper measures—a fixed, shared, hash-named generator in integer arithmetic that regenerates the same bytes on any implementation, block-parallel, selectable per object by a Manifest beside procedural, delta and retrieval engines—and a path to lower residue through engine size, since this engine is a thousandth of cmix’s. This paper claims no compression record on text.

Table 8: Stronger baselines on the two held-out books (ratio to raw; same byte strings as Table 7; bold = best without a prior). “+ dict” is **zstd -ultra -22** with a 148,024-byte dictionary trained on the engine’s 12 MB corpus; “| prior” is $|\text{compress}(\text{corpus} \parallel \text{book})| - |\text{compress}(\text{corpus})|$, the compressor given the engine’s corpus. Reproduced by **baselines.py**.

book	engine (float)	engine (fixed, C)	xz -9e	bz2 -9	brofli -11	zstd -22 + dict	PPMd o8	xz prior	PPMd prior
pg12.txt	0.274	0.284	0.316	0.279	0.305	0.298	0.252	0.267	0.221
pg84.txt	0.261	0.271	0.320	0.280	0.311	0.310	0.254	0.278	0.238

5.4 Domain engines, revisions and libraries on one disk

A production disk carries several engines and the Manifest’s GID selects one per object; at write time the disk tries every applicable engine and keeps the smallest Seed, with the escape always a candidate. Table 9 puts four on one disk: **ANET-EN**, the English engine; **ANET-CODE**, the same architecture trained on 2 MB of CPython source; **ANET-LIB**, a larger engine (hidden 384) trained on one document until its surprisal there is small; and **ANET-PROC**, a procedural engine—a Lorenz simulator (Lorenz, 1963) whose Seed is its parameter record and whose Manifestation reruns it.

Table 9: One disk, four engines. “ratio” is Seed/size with engines shared; “total” adds the chosen engine’s bytes, the number for a single-object disk. All bit-exact.

object	bytes	gzip	xz	engine chosen	Seed (B)	ratio	total	read (s)
lorenz.csv	1,153,112	506,927	362,792	ANET-PROC-lorenz-rk4-v1	118	0.00010	0.00010	0.1
pg12.txt	176,754	62,949	55,736	ANET-LIB	24,801	0.140	1.593	12.1
pg12_revised.txt	176,941	63,000	55,748	ANET-LIB	31,805	0.180	1.630	12.2
pg84_frankenstein.txt	200,000	76,855	68,228	ANET-EN	67,883	0.339	0.618	12.7
algorithmic_storage.py	20,439	6,544	6,408	ANET-CODE	9,475	0.464	3.190	1.4
image.png	88,791	83,598	82,608	VERBATIM	88,791	1.000	1.000	0.0

The simulation output regenerates from a 118-byte record (10^{-4} ; **xz**: 0.31). The library engine stores its document at $0.14\times$ against 0.32 for **xz**, but is larger than a one-document library, so the total is $1.59\times$; three more hours of training took the Seed to $0.094\times$ and the total nowhere, because the information merely moved from Seed to weights. Engine selection did the rest—the *Frankenstein* excerpt to ANET-EN ($0.34\times$; a favourable case, since that book is in the 55 KB engine’s training corpus, unlike in Table 7), Python source to ANET-CODE ($0.46\times$; **xz** 0.31), the PNG to the escape—without being told anything about domain.

Revisions cost their edits. Weights never change when an object is written, so a revision of an object already on the disk should not pay for it again. The *delta engine* makes a Seed a reference to a base object plus a token-level edit script (Ratcliff–Obershelp matching via **difflib**, Ratcliff and Metzener, 1988); the disk tries a delta against every existing text object, so lineage is

discovered rather than declared. Four versions of one document (Table 10): a 60-word revision costs 390 bytes ($0.0022\times$), a further revision with an appendix 390 bytes, a version with 5 % of all words changed 4,543 bytes; the four together occupy 30,124 bytes of Seeds against 223,328 for **xz** on each. Counting the engine: with the library engine the four cost 286,804 bytes, more than **xz**; with the shared 55 KB English engine storing the base they cost 126,136, 44 % below—the deltas are the same under either.

Table 10: Four versions of one document. Each was written with the library engine and, where smaller, as a delta against the best existing base. All bit-exact. Read times here were measured while a training job shared the machine’s single core; Table 9 has the same engine and object unloaded (12 s).

version	bytes	gzip	xz	engine	base	Seed (B)	ratio	read (s)
pg12.txt	176,754	62,949	55,736	ANET-LIB	—	24,801	0.140	28.7
rev1_60words.txt	176,941	63,000	55,748	ANET-DELTA	pg12.txt	390	0.00220	28.0
rev2_appendix.txt	178,561	63,087	55,884	ANET-DELTA	rev1_60words.txt	390	0.00218	28.7
rev3_5pct.txt	179,661	63,051	55,960	ANET-DELTA	rev2_appendix.txt	4,543	0.025	28.5
all four	711,917		223,328			30,124	0.0423	

Library documents cost a reference. Memorising a library with a neural engine is slow: The library engine was trained further at a constant learning rate and the document re-written to the disk every fifteen minutes (twelve checkpoints, 40–190 min of Work on the test machine). Its Footprint ratio fell from 0.140 to 0.099 (best 0.094 at 161 min) and then flattened, still well short of $0.01\times$: memorising a document into a 384-unit network is the wrong tool for that target, which the retrieval engine below reaches by construction (Figure 4). So the *retrieval engine* instead stores the library once as a content-addressed index of content-defined chunks (gear hash after FastCDC, Xia et al., 2016; mean 64 bytes) and predicts only what is not already there: a Seed is a layout of library runs interleaved with new segments, the new bytes coded by the English engine. The index is shipped once (104,252 B for a 331,237-byte, two-document library). Manifestation retrieves the runs, decodes the residual and assembles. A library document then costs 10 bytes—one run covering the whole document, regenerated in milliseconds from the index, the $0.01\times$ mark passed by two orders of magnitude by construction rather than by training (Table 11); a composite manual stitched from sixty paragraphs of two library documents plus new text, the case no other engine handles, $0.134\times$ (**xz**: $0.37\times$); for pure edits the delta engine still wins ($0.0022\times$ against $0.020\times$), because a changed word costs a whole chunk in a layout but only the word in an edit script; an unseen document finds no runs and costs exactly what the English engine costs. Retrieval never hurts, and the disk chose among four engines per object without being told anything about lineage or domain. It is de-duplication (Muthitacharoen et al., 2001; Tridgell and Mackerras, 1996) fused with a predictor: the cache corner of the triangle.

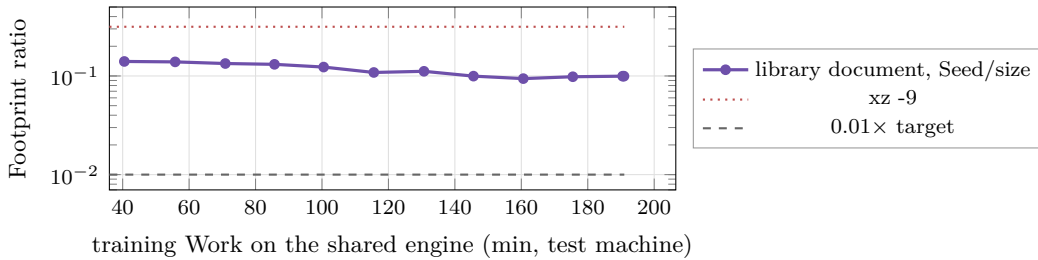


Figure 4: Footprint against training Work ($\mathcal{W}_{\text{train}}$): the library document’s Seed ratio at twelve checkpoints of the library engine, one core. The curve flattens near $0.1\times$; the information moves from Seed to weights, and the total does not fall.

Table 11: Retrieval engine beside the library, delta and English engines; every engine tried, the smallest Seed wins. All bit-exact. `rev3_5pct.txt` is a fresh random 5% revision generated for this experiment, hence its size differs from Table 10.

object	bytes	xz	LIB engine	delta	retrieval	runs	chosen	Seed (B)	ratio
<code>pg12.txt</code>	176,754	55,736	24,801	—	10	1	RET	10	0.00006
<code>rev1_60words.txt</code>	176,941	55,748	31,805	390	3,599	57	DELTA	390	0.00220
<code>rev3_5pct.txt</code>	178,127	55,856	48,792	4,082	36,779	526	DELTA	4,082	0.023
<code>composite_manual.txt</code>	26,188	9,764	9,649	12,366	3,502	59	RET	3,502	0.134
<code>pg84_frankenstein.txt</code>	200,000	68,228	67,883	69,510	67,894	0	EN	67,883	0.339

5.5 Determinism: one engine, two implementations, identical bytes

A Seed is only worth storing if it regenerates the same object on every device. A float engine cannot promise this: its probability tables depend on the order in which a dot product is summed, which differs between BLAS libraries, core counts and GPUs, and one differing table desynchronises the coder. The *fixed-point engine* removes the dependence by construction: Q12 integer weights, Q15 integer state, table-driven tanh and 2^x , int64 accumulators, integer soft-max—no floating point between Seed and object, obtained by post-hoc quantisation of the trained engine (Jacob et al., 2018). The harness encodes and decodes three objects under three ways of evaluating a dot product—plain, split-and-added (a different summation order), and float64 accumulation (a different precision)—and compares Seeds and tables against a reference (Table 12). The float engine fails every cross-variant case; the fixed-point engine passes all nine at essentially the same Footprint (10,035 against 10,046 bytes on the text). Its products stay below 2^{40} , so by construction any device with 64-bit integers should yield the same bits; what has been *verified* is three evaluation orders on one CPU and two independent implementations. The engine and coder were re-implemented in C (OpenMP over blocks): the C Seeds are byte-identical to the Python ones and each decodes the other’s, so the specification is complete; it runs $\sim 9\times$ faster (58 KB/s per core against 6.5), storing 3.4 MB of five books at 0.342 in 58 s (Table 13). The engine spends $\sim 49,000$ multiply–adds per byte, so throughput is set by arithmetic, not by the coder; the test machine’s single physical core gives block parallelism only $1.1\times$, and Horizon against cores needs a multi-core host.

Table 12: Cross-arithmetic determinism, three objects (text, Python, PNG). Cells are objects passing out of three; PASS = identical Seed and bit-exact regeneration of the reference Seed.

engine	variant	same Seed	tables identical	regenerates reference
float32	ref	3/3	3/3	3/3
float32	chunked	0/3	0/3	0/3
float32	f64	0/3	0/3	0/3
fixed-point	ref	3/3	3/3	3/3
fixed-point	chunked	3/3	3/3	3/3
fixed-point	f64	3/3	3/3	3/3

5.6 Deep data at scale, and a scaling check with 1000 files

Two objects of flight size were stored as procedural Manifests with generators written in *integer* arithmetic, bit-identical on any machine: 2.5 million records of synthetic attitude telemetry (a Lorenz system in Q20 fixed point standing in for a rigid-body simulation, 104.6 MB) and an 8192×8192 16-bit elevation model from hash-based integer value noise (fractal noise after Perlin, 1985; SplitMix64 mixing, Steele et al., 2014; 134.2 MB). Both regenerate bit-exactly (Table 14):

Table 13: Compiled fixed-point engine (hidden 128), blocks mode, by OpenMP thread count on a machine with one physical core and two hardware threads.

object	bytes	Seed (B)	ratio	xz	threads	read (s)	read KB/s
pg12.txt	176,754	60,223	0.341	0.315	1	3.04	58
					2	2.75	64
					4	2.73	65
five_books.txt	3,379,739	1,156,903	0.342	0.299	1	58.41	58
					2	52.67	64
					4	52.36	65

the telemetry costs 122 bytes of Seed plus 842 of program, 9×10^{-6} of its size, against 0.248 for **xz**, which also needed 170 s to compress it, fifteen times the 11 s the engine needs to regenerate it. The elevation model is the harder case for a compressor (0.552 for **xz**) and the same trivial case for the engine (1.2×10^{-5}). The disk file holding both Manifests is 895 bytes; with the two programs, 3.3 KB for 239 MB, a ratio of 7×10^4 (the decoder itself, a Python interpreter, is not counted, just as the **xz** binary is not). On a flight-class processor the telemetry regenerates in minutes and the model in a quarter of an hour—Horizons a probe on a months-long cruise does not notice. To be explicit about what this shows: both objects were produced by programs written for the experiment, so their Footprint is *provenance capture by construction*, not a discovery about the data. The experiment demonstrates the mechanism, the accounting and the Horizon at scale; its value in a mission is proportional to the share of data that is program output—all of simulation, rendering and synthetic products, none of raw sensor data.

 Table 14: Deep data at scale (Footprint counts Seed and program; **gzip -6/xz -6** because of the sizes; the flight-class column scales the measured time by ten and is indicative only—a RAD750/LEON-class processor may be thirty times slower).

object	bytes	Seed	program	Footprint ratio	gzip	xz	regen. (s)	MB/s	flight-class (s, est.)
telemetry.csv	104,632,381	122 B	842 B	9.2e-06	0.374	0.248	11	9.4	111
planet_dem.u16	134,217,728	78 B	1,591 B	1.2e-05	0.911	0.552	96	1.4	961

Scaling check with 1000 files. The Conclusion’s arithmetic was checked rather than asserted: one thousand distinct synthetic-telemetry runs were stored on a disk holding only the generator program and a catalogue of one thousand Manifests. Phase A generated each file once, recorded its length and sha-256 and discarded it (the moment at which, in a real system, the file was written); phase B regenerated every file from the catalogue *alone* and checked the digest; phase C queried the catalogue. Result: 1000 files, 104.5 GB of telemetry in all, stored as 123,907 bytes of Seeds plus a 1060-byte program; the catalogue file holding every Manifest is 315,075 bytes and the whole disk folder 317,228 bytes, $3.0\text{e-}06$ of the data. 1000 of 1000 files regenerated bit-exactly. Generating and hashing the thousand files took 3,836 s; regenerating and verifying them from the catalogue 3,640 s, 3.6 s per file on one core. The metadata query “rho > 30 and sigma < 10” was answered in 0.0002 s with 134 hits without regenerating anything. Like the two objects above, this is provenance capture by construction; what it adds is the scaling claim verified at a thousand objects, a catalogue that is the archive’s own index, and the batch Horizon a probe would pay.

5.7 Ten files through one disk

The released **demo/** folder runs the paradigm as a user would see it: ten files of 1–5 MB (four public-domain books never used to train any engine, three PDFs, two uncompressed procedural

animations and a NASA H.264 video) are written to a disk folder holding only engines, Seeds and Manifests, then regenerated from that folder alone and compared byte-for-byte. The disk carries the 256-unit English engine in compiled fixed-point form, the animation engine that made two of the videos, and the escape.

Table 15: Ten files through one AlgorithmicDisk. “engine/10” is the shared engine (201,006 B: the int8 English engine plus the animation generator’s source) divided by the ten files that share it; “Seed + engine/10” is what each file effectively occupies; “output” is the size of the regenerated file, byte-identical to its input in every row. **xz -6** for comparison.

file	input (B)	engine	Seed (B)	engine/10	Seed+eng./10	output (B)	ratio	xz
animation_A.avi	3,457,672	PROC	110	20,101	20,211	3,457,672	0.006	0.446
animation_B.avi	2,700,952	PROC	110	20,101	20,211	2,700,952	0.007	0.430
brothers_karamazov.txt	2,039,159	EN-256	559,018	20,101	579,119	2,039,159	0.284	0.285
cinderella_storybook.pdf	1,055,953	raw	1,055,953	20,101	1,076,054	1,055,953	1.019	0.470
count_of_monte_cristo.txt	2,787,124	EN-256	761,721	20,101	781,822	2,787,124	0.281	0.279
don_quixote.txt	2,391,678	EN-256	701,402	20,101	721,503	2,391,678	0.302	0.287
nasa_mercury_transit.mp4	3,413,359	raw	3,413,359	20,101	3,433,460	3,413,359	1.006	0.985
quixote_text_compressed.pdf	1,205,694	raw	1,205,694	20,101	1,225,795	1,205,694	1.017	0.920
quixote_text_uncompressed.pdf	4,012,832	raw	4,012,832	20,101	4,032,933	4,012,832	1.005	0.132
wealth_of_nations.txt	2,468,951	EN-256	708,399	20,101	728,500	2,468,951	0.295	0.219
all ten	25,533,374		12,418,598	201,006	12,619,604	25,533,374	0.494	0.422

Read the table as a skeptic would. Across the ten, Seeds plus engine occupy 0.494 of the input (0.495 with the Manifests) against 0.422 for **xz -6**: on this mixed set the disk *loses* to a free, standard compressor, because four files sit at $1.000\times$ —a PDF of page images, a Flate-compressed PDF, an H.264 video and a PDF whose text is hex-encoded in its content streams—for which no engine on this disk exists, so the escape keeps their bytes rather than expand them. The four unseen books, at $0.27\text{--}0.29\times$, are on par with **xz**: creditable for a 148 KB engine, but parity, not a breakthrough. The two animations, 3.5 and 2.7 MB stored as 110-byte parameter records, are the only dramatic rows, and they are provenance capture: every pixel of every frame is a deterministic function of seven numbers, the file carries about 110 bytes of information and 3.4 MB of the program’s doing, and **xz**, which looks for repeated bytes rather than for the program, reaches only 0.446 . Point a camera at the sky and record the same 3.4 MB and no program on any disk produces those bytes from 110 numbers—which is why the NASA video in the same table sits at $1.000\times$. What the table demonstrates is the paradigm working end to end and the law of Section 2 visible in one place; it does not demonstrate superiority over existing compression on general files, and this paper does not claim it. The remedy for the engine-less files is more engines, which is a research programme rather than a result.

5.8 What the numbers mean

Why shared weights do not make every file cost $0.01\times$. The Seed is the part of the object the engine cannot predict. Shared weights remove the *shared* part—for English, the spelling, grammar and common phrases—which is why an unseen book’s Seed is smaller than **xz**. What remains is the object’s own novelty: which words this author chose. That residue is about 2.1 bits per character for the engine of Table 7, 2.0 for PPMd, about 1.2 for cmix and NNCP on Wikipedia text ($0.15\times$), about 0.7 for a 70-billion-parameter language model used as a coder with its size not counted (Delétang et al., 2023), and Shannon put the entropy of English near one bit per character (Shannon, 1951), $\approx 0.12\times$; $0.01\times$ is 0.08 bits per character, below the information content of the text by any of these estimates. No weights, however good or widely shared, take a *novel* object below its own information content; that is the floor, and it holds for every storage

scheme. The *weight* cost per object does go to zero as objects are added (Table 5); the Seed cannot. So the per-object Footprint is tiny exactly when the object is *not* new to the disk: program output (10^{-4} – 10^{-6}), a revision (2×10^{-3}), a library document (6×10^{-5}).

Why not one engine per file. An engine shared by nothing is part of that file’s Footprint, and engine plus Seed must then hold the file’s whole information: compression by another name, under the same floor. The single-document library engine measured it—Seed 24,801 B, engine 256,680 B, $1.59\times$ —and further training moved information from Seed to weights without moving the total; a network memorising bytes is a worse compressor than **xz**, as implicit neural representations show for images. The saving comes only from sharing: put the domain’s knowledge in the engine, and every object pays only for what is genuinely its own.

Design rules learned. The demo engine reached its first bit-exact object only on its sixth revision, and the failures are rules for any AlgorithmicNET. (1) Inject the Seed at every step through a context-dependent projection: a Seed used only as $\mathbf{h}_0$ decays into a fixed point (180 of 232 bits), and one injected identically at every step ties all blocks together (195 of 232). (2) Stay out of saturation: a soft-plus loss drove 87 % of units into saturation, where gradients vanish; a hinge loss with the Seed projected onto a norm ball fixed it. (3) Accept only *quantised* Seeds that reproduce every bit with a margin. (4) Give the engine output feedback, or every object is a memorisation problem and the exchange rate stays near one. (5) Prefer closed-form discovery where it exists.

6 The Device, Deployment and Policy

The AlgorithmicDisk of Section 5 is a file and a program; as a device it is a sealed unit with one input, a Manifest, and one output, the object, streamed serially and bit-exactly.

- **Input: a Regeneration Manifest**—generator identity, Seed, length, digest, block length, mode; a few bytes to a few kilobytes, arriving over any narrow link: a serial line, a radio frame, a QR code, a hand-typed string.
- **Output: the object, in order, as a byte stream** on a serial port, a USB endpoint, a socket, or directly into a consumer (a display, a printer, a guidance computer); the first byte is available after one engine step and the stream ends with a verified digest.
- **Control:** a *materialise* command (write the stream to the cache), an *evict* command, and a status line reporting the Horizon the device needs for a given Manifest at its Compute.

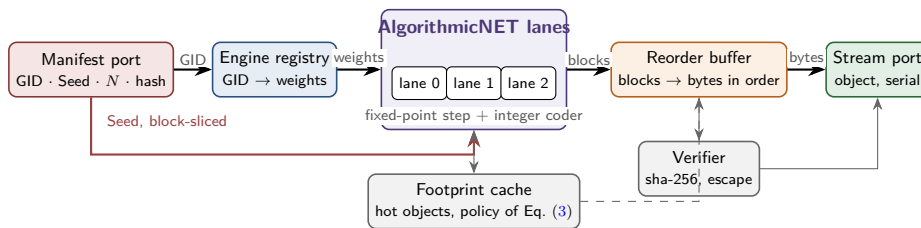


Figure 5: The Regeneration Appliance. A Manifest enters; the registry selects the engine by GID; the Seed is sliced into block streams and fed to p identical fixed-point lanes; a reorder buffer restores object order; the verifier checks the digest; the object leaves as a serial byte stream. The cache holds materialised objects under the regenerate-versus-cache policy.

Engine registry: non-volatile memory with one integer weight set per generator identity (55–257 KB per engine here), so a registry of domain engines is a few megabytes of flash and every

appliance regenerates the same bytes from the same Manifest. **Lanes:** p copies of the fixed-point step and integer coder, each owning one block from $\mathbf{h} = \mathbf{0}$ with its block code and slice of the Seed; no floating point, 4 KB of state. **Slicer and reorder buffer:** block j goes to lane $j \bmod p$ and a $p \times L$ -byte buffer releases bytes in object order. **Verifier and escape:** the digest is checked as the stream leaves; verbatim blocks bypass the lanes. **Cache:** optional RAM or flash under the policy below. Procedural and delta engines are lane programs. The lane cost is fixed by the engine ($\sim 5 \times 10^4$ multiply–adds per byte for the 128-unit engine), so the appliance’s Horizon is arithmetic (Table 16); the rows are the triangle in hardware—same Manifest, same bytes, a Horizon that shrinks as Compute is added. The appliance’s Footprint is its registry plus its Manifests—every engine of this paper and a million Manifests is a few hundred megabytes—while its apparent capacity is the sum of the objects they regenerate.

Table 16: Streaming rate against Compute, 128-unit engine. Only the first row is measured; the others are illustrative upper bounds from the multiply–add count and assume the lanes are kept busy, which a sequential recurrence does not guarantee on wide hardware.

realisation	Compute	lanes	rate	100 MB object
this paper, one x86 core, C	~ 3 GMAC/s eff.	1–2	58 KB/s (meas.)	29 min
microcontroller (Cortex-M7 class)	0.2 GMAC/s (est.)	1	4 KB/s	7 h
FPGA, 64 DSP lanes at 200 MHz	13 GMAC/s (est.)	64	250 KB/s	7 min
GPU module (int8, 10^4 lanes)	10 TMAC/s nominal	10^4	≤ 200 MB/s	≥ 0.5 s

Where the trade pays, and where it does not. On Earth a byte at rest is nearly free: flash costs of the order of a cent per gigabyte, and a microSD card holds more than every object in this paper. Regenerating a file over hours to save such bytes is never worth it, and the paper does not propose it. The trade pays where the scarce quantity is not the byte at rest but the byte *in transit* over a narrow link (a deep-space uplink of kilobits per second, a text message), the byte that must *survive* a medium or a lifetime that flash cannot, or the verification that what arrived is exactly what was sent; the Manifest is the unit of all three. The regimes below are chosen on that criterion.

Deployments (Table 17). An interplanetary probe pays in Horizon: manuals, maps and model updates regenerated over months of cruise from Manifests of a few hundred bytes, uplinked from a ground segment that keeps the same registry and discovers Seeds with data-centre Compute. A solar outpost regenerates on demand, deletes what is idle and recreates it later, using sunlight as storage. A millennial archive keeps centuries of records in a small vault, accepts hours per retrieval, and is queried through its catalogue of Manifests without regenerating anything. A school without internet has the shape of a probe—a narrow, intermittent link (a phone with SMS, a community radio, a USB stick carried by a visitor) and patient users. The appliance is a single-board computer or a solar laptop carrying the engine registry and a catalogue in which every textbook, lecture and dataset is a Manifest of a few hundred bytes to a few kilobytes, so a whole curriculum’s catalogue fits in a feature phone’s memory; the value is the link, not the storage it saves. A revised chapter is its edits alone and a new title a Seed against the engines already on site, so a term’s updates arrive in one text message or one radio pass. Content is created on demand from the AlgorithmicNET and the Seeds: a teacher deletes last month’s chapter to make room for the next one, starts its regeneration the day before while the panel charges, and finds it in the cache for class; the deleted chapter loses only its bytes and returns from its Seed when needed; and the digest in every Manifest tells the teacher that what the students read is exactly what was sent. A remote spacecraft is the same design at a greater distance: it restores whatever it needs from the condensed source it carries—engines, Seeds, Manifests—and discards the bytes

when done. For deep space the lane array is a radiation-tolerant FPGA, the registry lives in flight flash, and the ground, because the engine is fixed-point, verifies before uplink the exact bytes the probe will produce. The data centre is where Seed Discovery for all of the above is done.

Table 17: Operating points on the Regeneration Triangle (“↑” abundant, “↓” scarce).

Deployment	$\mathcal{F}$	$\mathcal{C}$	$\mathcal{H}$	Pays with	Example
Interplanetary probe	↓	↓	↑↑	Horizon	manuals, maps, models regenerated over months of cruise
Planetary outpost	↓	↑ (solar)	↑	Compute × Horizon	regenerate on demand, delete when idle, recreate later
Millennial archive	↓↓	↓	↑↑↑	Horizon	centuries of records in a small vault; hours per retrieval
Palm-scale device	↓	↓	medium	amortised weights	one shared engine, thousands of Seeds
Off-grid school	↓	medium	↑	Horizon + engines	one solar board holds a whole library as Manifests
Data centre	↑	↑↑	↓	Compute	Seed Discovery for all of the above

Regenerate-versus-cache policy. Space is bought once; time is paid on every read. A stored byte costs its space for as long as it exists, whereas a regenerated object costs its Horizon each time it is read, so the space-for-time conversion is a rental rather than a purchase. With $c_{\mathcal{F}}$ the cost per byte-second of storage and $c_{\mathcal{W}}$ the cost per operation, an object accessed λ times per second should be cached iff

$$\lambda > \lambda^* = \frac{c_{\mathcal{F}} N}{c_{\mathcal{W}} \mathcal{W}_{\text{read}}(N)}, \quad (3)$$

the rate at which the device decides, object by object, whether to rent time or buy space. On a solar-powered node $c_{\mathcal{W}}$ falls to nearly zero in daylight, driving λ^* up and the resident cache down: the system uses sunlight as storage. Eq. (3) is the design rule for experiment E4 and has not yet been measured.

7 Open Experiments and Future Work

The experimental programme and its status are in Table 18; a GPU run of the determinism harness is the sixth open item.

Table 18: Experimental programme. Each measures one exchange rate of the triangle.

Exp.	Name	Measures	Data	Status
E1	Frontier sweep	$\mathcal{F}^*(\mathcal{W})$ per data class under increasing Work budgets	simulation, telemetry, images, text, random	planned
E2	Seed Discovery	Footprint vs. write-time Work for the discovery strategies	232-bit object; procedural data	Table 4 (miniature)
E3	Amortisation	per-object Footprint vs. corpus size M with a shared engine	Alice corpus; whole books	Tables 5–11
E4	Policy	realised cost of Eq. (3) on a solar power trace	outpost workload replay	planned
E5	Substitution	Horizon vs. cores for block-parallel manifestation	a 1 TB object	compiled coder built; needs a multi-core host

Hypotheses and status. (H1) On deep data $\mathcal{F}^*(\mathcal{W})$ falls by orders of magnitude at modest Work—*supported* (Tables 9 and 14). (H2) Amortised per-object Footprint decreases monotonically with M —*confirmed* (Table 5). (H3) Manifestation Horizon scales as $1/p$ in the number of cores p —*not yet testable* on a single-core machine. (H4) The policy of Eq. (3) reduces resident Footprint by $\geq 90\%$ on an outpost replay at no loss of availability—*planned*.

Parallel manifestation. Within a block every engine here emits its bits in series, and serial emission is what makes Horizon the price of Footprint. Three layouts are proposed. *Segment* parallelism, the blocks mode already built, costs a few percent of Seed and one flush per segment. *Interleaved* (strided) parallelism has lane ℓ of p emit bits $\ell, \ell+p, \ell+2p, \dots$, so each step across the lanes is one contiguous word of the object, streamed in order with no reassembly; the lanes share the Seed injection and the block code, and for a predictive engine the open question is making them conditionally independent given the shared state, by a bit-plane or residue-class factorisation of the next-symbol distribution. *Hierarchical* parallelism has a coarse engine emit a sketch and fine engines fill the gaps in parallel; the serial path would shrink from N to $O(N/2^k + k)$ if the fine engines can be conditioned on sketch points without loss, an untested assumption. In every layout only the order of emission changes—the practical form of Proposition 2.1(iv).

8 Related Work and Attribution

The backbone is Kolmogorov complexity, its resource-bounded variants and Bennett’s logical depth; cryptanalytic time–memory trade-offs (Hellman, 1980) are the classical template for an exchange law between storage and computation, and the triangle adds time as a separately budgeted side and applies the trade to data at rest. “Algorithmic networks” (Abrahão et al., 2017) is an unrelated construct; the name AlgorithmicNET is not derived from it. As a coder the predictive engine is the classical model-plus-entropy-coder construction (Witten et al., 1987; Martin, 1979), of which PPM (Cleary and Witten, 1984; Shkarin, 2002) is the context-model form that Table 8 shows still beats the engine, and cmix (Knoll, 2014), NNCP (Bellard, 2019) and Delétang et al. (2023) the neural form, reaching 1.2 bits per byte and below with models thousands of times larger than the 55–148 KB engines here; shared static or trained dictionaries (Alakuijala and Szabadka, 2016; Collet and Kuchera, 2021) are the existing form of a prior amortised across objects; integer-only networks for cross-platform deterministic entropy coding were introduced by Ballé et al. (2019), and the fixed-point engine of Section 5.5 applies the same principle to a recurrent byte predictor; this paper does not compete with them on ratio but uses the construction as one engine among several inside a storage device, adding the Manifest, engine selection, the escape, the determinism guarantee and the deployment model, and measuring the trade the device makes rather than the compression a model achieves. Implicit neural representations (Dupont et al., 2021; Sitzmann et al., 2020; Mildenhall et al., 2020) and auto-decoders (Park et al., 2019) are the relatives of the latent-Seed design; content-defined chunking (Muthitacharoen et al., 2001; Xia et al., 2016) and delta transfer (Tridgell and Mackerras, 1996) are the ancestors of the retrieval and delta engines; rematerialisation (Chen et al., 2016) is the same trade inside one training run; integer-only inference (Jacob et al., 2018) is used here for the property Ballé et al. (2019) identified, identical output across devices. The seed-driven, inverse-manifestation view is shared with the EmbryonicAL model (Patil, 2026): it grows a *model* from a Seed; the AlgorithmicNET grows *data*. Landauer’s principle (Landauer, 1961) bounds the energy side of Compute; reversible computing (Bennett, 1973) suggests the energy cost of manifestation, unlike its Footprint, has no floor. Nothing in the implementation’s building blocks is claimed as new: Subbotin’s carry-less range

coder, FastCDC’s gear hash, Ratcliff–Obershelp matching, teacher forcing, the straight-through estimator, Adam, the Lorenz system, fractal value noise, SplitMix64, NumPy (Harris et al., 2020) and Project Gutenberg (Project Gutenberg, 2026) are all credited where used.

9 Limitations and Final Notes

The evidence is narrower than the vision. The engines are small and the text engines know only English, and on novel text they sit among the standard compressors rather than above them (Table 8); the deep-data objects were generated by programs written for the experiment, so those ratios demonstrate provenance capture rather than a property of mission data; determinism has been verified across arithmetic variants on one CPU, not yet on a GPU; Horizon against cores could not be measured on a single-core machine; the caching policy is unmeasured. Two structural limits matter more than any number. First, a Manifest is worthless without its engine: the disk concentrates a whole library’s survival in a few hundred kilobytes of weights that must be preserved, replicated and kept executable—a single point of failure raw files do not have, and the first thing an archive design must address. Second, the information floor: for novel data the paradigm buys modest compression at large computational cost; only data predictable given what the disk already holds buys orders of magnitude. What remains—the GPU run, Horizon against cores, the emission schedules, the policy on a real power trace—needs only hardware this study did not have; the released code lets anyone run them and check every result here.

Code and test cases: <https://github.com/syncaissa>.

10 Conclusion

Storage has always been paid for in bytes. This paper measures how far it can be paid for in computation and time instead, under a law with a floor: an object’s Footprint can fall to what the disk cannot already predict, and no further. An AlgorithmicDisk holds engines and Seeds rather than files: the engine’s parameters are shared by every object, so an object costs its own Seed plus an ever-smaller fraction of the engine as more objects are stored with different Seeds—the more, the better. One unseen book costs the 148 KB engine plus a 48 KB Seed, more than **xz**; a thousand cost 48.5 KB each, 13 % less than **xz** and 8 % more than PPMd needs with no engine at all—the engine is PPM-class, and the paper claims no compression record. A thousand 105 MB program-generated telemetry files—104 GB—were stored in 317 KB and every one regenerated bit-exactly: provenance capture, which is what the paradigm is for, not compression of arbitrary data. Every object regenerates bit-exactly on any device; revisions cost their edits; 239 MB of simulation output came from 3.3 KB. The gains are real exactly where the object has structure an engine can hold—computational for deep data, statistical for a domain, shared content for a library—and the exchange rate is the engine’s knowledge of that structure. That is the practical content of Proposition 2.1, and it is why a device that stores Seeds instead of files is not a compression trick but a different place to stand on the Regeneration Triangle.

References

- | | |
|---|--|
| <p>Abrahão, F. S., Wehmuth, K., and Ziviani, A. (2017). Algorithmic networks: central time to trigger expected emergent open-endedness. <i>arXiv:1708.09149</i>.</p> <p>Alakuijala, J. and Szabadka, Z. (2016). Brotli compressed data format. Technical Report RFC 7932, IETF.</p> | <p>Ballé, J., Johnston, N., and Minnen, D. (2019). Integer networks for data compression with latent-variable models. In <i>ICLR</i>.</p> <p>Bellard, F. (2019). Lossless data compression with neural networks. https://bellard.org/nncp/.</p> <p>Bengio, Y., Léonard, N., and Courville, A. (2013). Estimating or propagating gradients through stochastic</p> |
|---|--|

- neurons for conditional computation. *arXiv:1308.3432*.
- Bennett, C. H. (1973). Logical reversibility of computation. *IBM Journal of Research and Development*, 17(6):525–532.
- Bennett, C. H. (1988). Logical depth and physical complexity. In Herken, R., editor, *The Universal Turing Machine: A Half-Century Survey*, pages 227–257. Oxford University Press.
- Bernstein, D. J. (2008). ChaCha, a variant of Salsa20. Technical report, Workshop Record of SASC 2008: The State of the Art of Stream Ciphers.
- Chaitin, G. J. (1966). On the length of programs for computing finite binary sequences. *Journal of the ACM*, 13(4):547–569.
- Chen, T., Xu, B., Zhang, C., and Guestrin, C. (2016). Training deep nets with sublinear memory cost. *arXiv:1604.06174*.
- Cleary, J. G. and Witten, I. H. (1984). Data compression using adaptive coding and partial string matching. *IEEE Transactions on Communications*, 32(4):396–402.
- Collet, Y. and Kucherawy, M. (2021). Zstandard compression and the application/zstd media type. IETF RFC 8878.
- Cover, T. M. (1965). Geometrical and statistical properties of systems of linear inequalities with applications in pattern recognition. *IEEE Transactions on Electronic Computers*, EC-14(3):326–334.
- Delétang, G., Ruoss, A., Duquenne, P.-A., Catt, E., Genewein, T., Mattern, C., Grau-Moya, J., Wenliang, L. K., Aitchison, M., Orseau, L., Hutter, M., and Veness, J. (2023). Language modeling is compression. *arXiv:2309.10668*.
- Dupont, E., Goliński, A., Alizadeh, M., Teh, Y. W., and Doucet, A. (2021). COIN: COmpression with implicit neural representations. *arXiv:2103.03123*.
- Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. *Nature*, 585:357–362.
- Hartmanis, J. (1983). Generalized Kolmogorov complexity and the structure of feasible computations. In *24th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 439–445.
- Hellman, M. E. (1980). A cryptanalytic time-memory trade-off. *IEEE Transactions on Information Theory*, 26(4):401–406.
- Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., and Kalenichenko, D. (2018). Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *CVPR*.
- Kingma, D. P. and Ba, J. (2015). Adam: a method for stochastic optimization. In *ICLR*.
- Knoll, B. (2014). cmix. <https://www.byronknoll.com/cmix.html>.
- Kolmogorov, A. N. (1965). Three approaches to the quantitative definition of information. *Problems of Information Transmission*, 1(1):1–7.
- Landauer, R. (1961). Irreversibility and heat generation in the computing process. *IBM Journal of Research and Development*, 5(3):183–191.
- Levin, L. A. (1973). Universal sequential search problems. *Problems of Information Transmission*, 9(3):265–266.
- Li, M. and Vitányi, P. M. B. (2008). *An Introduction to Kolmogorov Complexity and Its Applications*. Springer, 3rd edition.
- Lorenz, E. N. (1963). Deterministic nonperiodic flow. *Journal of the Atmospheric Sciences*, 20(2):130–141.
- Martin, G. N. N. (1979). Range encoding: an algorithm for removing redundancy from a digitised message. In *Video & Data Recording Conference*, Southampton.
- Mildenhall, B., Srinivasan, P. P., Tancik, M., Barron, J. T., Ramamoorthi, R., and Ng, R. (2020). NeRF: Representing scenes as neural radiance fields for view synthesis. In *ECCV*.
- Muthitacharoen, A., Chen, B., and Mazières, D. (2001). A low-bandwidth network file system. In *Proceedings of the 18th ACM Symposium on Operating Systems Principles (SOSP)*, pages 174–187.
- Park, J. J., Florence, P., Straub, J., Newcombe, R., and Lovegrove, S. (2019). DeepSDF: Learning continuous signed distance functions for shape representation. In *CVPR*.
- Patil, M. K. (2026). EmbryonicAI model: Seed-driven neural network growth for regenerative AI. *Syncaissa Systems Inc. Technical Report*. <https://github.com/syncaissa/EmbryonicAI>.
- Perlin, K. (1985). An image synthesizer. *ACM SIGGRAPH Computer Graphics*, 19(3):287–296.
- Project Gutenberg (2026). Project gutenberg literary archive. <https://www.gutenberg.org>.
- Ranzato, M., Chopra, S., Auli, M., and Zaremba, W. (2016). Sequence level training with recurrent neural networks. *ICLR*.
- Ratcliff, J. W. and Metzener, D. E. (1988). Pattern matching: the gestalt approach. *Dr. Dobbs' Journal*, 13(7):46.
- Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal*, 27:379–423, 623–656.
- Shannon, C. E. (1951). Prediction and entropy of printed English. *Bell System Technical Journal*, 30(1):50–64.
- Shkarin, D. (2002). PPM: one step to practicality. In *Data Compression Conference (DCC)*, pages 202–211.
- Sitzmann, V., Martel, J. N. P., Bergman, A. W., Lindell, D. B., and Wetzstein, G. (2020). Implicit neural representations with periodic activation functions. In *NeurIPS*, volume 33.
- Solomonoff, R. J. (1964). A formal theory of inductive inference. Part I. *Information and Control*, 7(1):1–22.
- Steele, G. L., Lea, D., and Flood, C. H. (2014). Fast splittable pseudorandom number generators. In *OOPSLA*, pages 453–472.
- Subbotin, D. (1999). Carryless rangecoder. Code and notes circulated on comp.compression.
- Tridgell, A. and Mackerras, P. (1996). The rsync algorithm. Technical Report TR-CS-96-05, Australian National University.
- Williams, R. J. and Zipser, D. (1989). A learning algorithm for continually running fully recurrent neural networks. *Neural Computation*, 1(2):270–280.
- Witten, I. H., Neal, R. M., and Cleary, J. G. (1987). Arithmetic coding for data compression. *Communications of the ACM*, 30(6):520–540.
- Xia, W., Zhou, Y., Jiang, H., Feng, D., Hua, Y., Liu, Y., and Zhang, Y. (2016). FastCDC: a fast and efficient content-defined chunking approach for data deduplication. In *USENIX Annual Technical Conference*, pages 101–114.